

Posto: Probabilistic Safety Monitoring — User Guide

What Posto Does

Posto lets you check if a dynamical system remains safe under uncertainty by:

1. Simulating trajectories

- **Equation mode:** next state from equations in a JSON file.
- **ANN mode:** next state from a trained neural network (`.h5`).

2. Generating uncertain logs

- From a single simulated trajectory.
- Each logged value is replaced by an interval `[lo, hi]` of fixed half-width `--dtlog`.
- Only a random subset of time steps (probability `--prob%`) are logged, plus always `t = 0`.

3. Checking safety

- Safety constraints are read from a JSON file (`safety_constraints`).
- Both **log samples** and **simulated trajectories consistent with the log** are checked.
- Output is SAFE / UNSAFE plus plots.

Installation & Environment

Python Dependencies

Install the libraries actually used by this codebase:

```
pip install numpy matplotlib docopt tensorflow
```

You can also use keras directly instead of tensorflow; the code tries:

```
from tensorflow.keras.models import load_model
# falls back to:
from keras.models import load_model
```

Environment Variable: POSTO_ROOT_DIR

Many modules expect the project root via:

```
PROJECT_ROOT = os.environ[ 'POSTO_ROOT_DIR' ]  
sys.path.append( PROJECT_ROOT )
```

Set it once:

```
export POSTO_ROOT_DIR=/absolute/path/to/Posto
```

Add this export to your `~/.bashrc` file to make it permanent.

Repository Layout

```
Posto/  
├─ posto.py  
├─ System.py  
├─ Parameters.py  
├─ artEval.py  
├─ artEvalNN.py  
├─ art/  
│   ├── figA2a  
│   │   └─ img  
│   ├── figA2b  
│   │   └─ genTimePlot.py  
│   ├── figA2c  
│   │   └─ confidence.py  
│   ├── figA3a  
│   │   └─ img  
│   ├── figA3b  
│   │   └─ img  
│   ├── figA3c  
│   │   └─ img  
│   ├── figA3d  
│   │   └─ img  
│   ├── figA4a  
│   │   └─ img  
│   ├── figA4b  
│   │   └─ img  
│   ├── figA4c  
│   │   └─ img  
│   ├── figA4d  
│   │   └─ img  
│   ├── figB5  
│   │   └─ img  
│   ├── figB6a  
│   │   └─ img  
│   └─ figB6b
```



```
{
  "state_vars": ["x", "y"],
  "constants": {"dt": 0.01},
  "ranges": {"ep": [-0.002, 0.002]},
  "equations": {
    "x'": "x + dt * (-y - (1.5*(x*x) - 0.5*(x*x*x) - 0.5)) + ep",
    "y'": "y + dt * ((3*x) - y) + ep"
  },
  "safety_constraints": [
    { "state": "x", "op": "le", "const": -0.10 },
    { "state": "y", "op": "le", "const": -0.10 }
  ]
}
```

Safety Constraints Semantics

Unsafe whenever constraint evaluates *true*:

- "ge": state $\geq$ c
- "le": state $\leq$ c
- "gt": state $>$ c
- "lt": state $<$ c

Command-Line Interface (posto.py)

Commands:

```
posto.py behavior
posto.py generateLog
posto.py checkSafety
```

Global Arguments

- --log=<logfile>
- --mode=<equation|ann>
- --model_path=<path>
- --states=<comma-list> (optional, required for ANN)
- --constraints=<json> (optional, required for ANN safety-check)

behavior

Simulates multiple trajectories and saves plots under `logdir/img/`.

generateLog

Creates a log `.lg` file with interval uncertainty plus visualizations.

checkSafety

Performs safety classification:

1. Check log samples
2. Sample valid trajectories consistent with the log
3. Jeffreys Bayes Factor decides SAFE vs UNSAFE

Plots saved under `logdir/img/`.

Custom / Dev Mode

`dev/Model.py` shows how to override `getNextState()` manually.

Constraints can be provided as Python tuples.

In-Depth: Using `Model.py` for Custom Dynamics

Sometimes you want full Python control (algorithmic logic, conditionals, calls to third-party libs) instead of JSON equations. The `Model.py` path lets you **override** the system's next-state map at runtime.

Minimal custom model

Model.py

```
# Model.py
import random

def my_getNextState(state):
    """
    state: tuple/list of floats, e.g., (x, y)
    return: next-state tuple of same length
    """
    x, y = state
    # simple drift with tiny noise
    x = x + 0.1 + random.uniform(-0.01, 0.01)
    y = y - 0.05 + random.uniform(-0.01, 0.01)
    return (x, y)
```

Use it with System

```
# examples/run_with_model_py.py
from System import System
from Model import my_getNextState

# no mode/model_path: we'll inject our own step function
sys = System(log_path="logs/custom_with_modelpy.lg", mode=None, model_path=None)

# override the engine step
sys.getNextState = my_getNextState

# generate a log: init set must match state dimension (x, y)
init_box = [[0.0, 1.0], [0.0, 2.0]]
sys.generateLog(init_box, T=100, prob=50, dtlog=0.01)
```

Run

```
python examples/run_with_model_py.py
```

Custom model with 3 states (x, y, z) and parameters

```
# Model.py
import random
from System import System
ALPHA, BETA, GAMMA, DT = 2.0, 0.5, 1.0, 0.01

def get_noise(lo, hi):
    return random.uniform(lo, hi)

def my_getNext(state):
    """A Python equivalent to the JSON equations"""
    x, y, z = state
    ep_x = get_noise(-0.005, 0.005)
    ep_y = get_noise(-0.002, 0.002)
    ep_z = get_noise(-0.003, 0.003)

    x_next = x + DT * (ALPHA * y - BETA * x**3) + ep_x
    y_next = y + DT * (BETA * x**2 - GAMMA * z) + ep_y
    z_next = z + DT * (x - y + GAMMA * z**2) + ep_z
    return (x_next, y_next, z_next)
```

Plug it into `System`

```
my_states = ['x', 'y', 'z']
my_constraints = [(1, 'ge', 0.49), (2, 'le', -0.10) ]
sys = System(log_path="logs/custom_with_modelpy.lg", states=my_states,
constraints=my_constraints)

# override the engine step
sys.getNextState = my_getNextState

# generate a log: init set must match state dimension (x, y, z)
init_box = [[0.0, 0.2], [0.0, 0.2], [0.0, 0.2]]
sys.behaviour(init_box, T=1000)
sys.generateLog(init_box, T=2000, prob=2, dtlog=0.02)
sys.checkSafety()
```

Run

```
python dev/Model.py
```